

📌 Objectifs de la feuille

- ORM DJANGO
- Création du modèle
- Exploration des champs du modèle
- Gestion de la base de données avec les migrations
- Test du modèle
- Shell interactif Python et API DJANGO
- Mise à jour de la vue pour afficher le modèle

Représenter une entité avec un modèle

Quel que soit le but de votre application web, il est très probable qu'elle doive stocker des données. Les programmeurs commencent souvent par identifier les différentes entités au sujet desquelles nous devons stocker des données.

Pour chaque entité pour laquelle nous voulons stocker des données, nous créons un modèle pour représenter cette entité. Un modèle définit les caractéristiques que nous voulons stocker à propos d'une entité particulière.

Notre modèle de groupe pourrait donc avoir des caractéristiques. Ces caractéristiques sont également connues sous le nom de champs. Nous pouvons ensuite utiliser le modèle pour créer des objets individuels, ou instances, de ce modèle, qui ont chacun leurs propres caractéristiques uniques. On pourrait penser que le terme «modèle» n'est qu'un autre nom pour une «classe» ! Et vous auriez à moitié raison : vous créez un modèle dans DJANGO en définissant une classe Python.

En général, dans les frameworks MVC et MVT, un modèle est également capable de stocker (ou de "persister") ses données dans une base de données pour une utilisation ultérieure. Cela contraste avec les classes et objets ordinaires, dont les données existent temporairement : par exemple seulement pendant l'exécution de l'application. De même, les «caractéristiques» des classes Python sont appelées attributs, mais lorsqu'un modèle enregistre un attribut dans la base de données, il s'agit d'un champ.

C'est quoi un ORM?

Le plus dans DJANGO c'est son ORM, très puissant et très simple à utiliser. Bien sûr il ne répondra pas à tous vos besoins : passer par des requêtes SQL est inévitable mais 99,99% du temps vous avez un outil qui vous assiste vraiment.

ORM est l'acronyme anglais de Object Relational Mapping , donc mapping d'objet relationnel en français. Un ORM est une technique de programmation qui donne l'illusion de travailler avec une base de données orientée objet . Pour résumer vous ne faites plus de requêtes SQL mais vous travaillez directement avec vos objets.

L' ORM de DJANGO est très bien conçu. D'une part il vous permet de lister tous les objets nécessitant des enregistrements (on parle de modèles) dans un endroit prévu à cet effet. Si vous êtes un nouveau développeur sur un projet, vous comprenez ce projet en quelques secondes en visualisant ces fichiers. Un autre avantage de travailler avec un ORM, c'est qu'il est une couche d'abstraction qui exploite la base de données ce qui fait que le développeur ne se soucie plus du SGBDR que le projet utilise . Comme nous l'avons vu précédemment concernant les bases de données, utiliser SQLite ou MySQL implique des requêtes SQL spécifiques dans certains cas. Avec Django , il n'y a pas de différence, c'est lui qui gère de faire la requête compatible en fonction du SGBDR. La compatibilité des bases de données, c'est devenu son boulot, plus le vôtre.

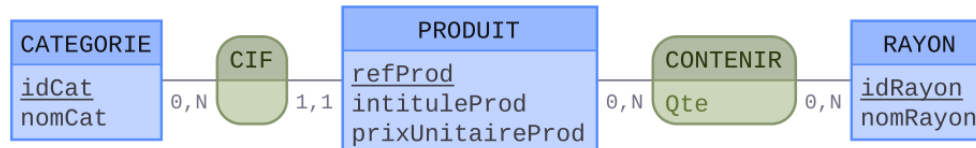
Un modèle est donc pour faire simple une table de données. On lui donne un nom, des champs typés et un comportement. Travailler avec un ORM permet donc une homogénéité du code , une structure plus solide , plus simple à maintenir et est optimisé pour les requêtes les plus simples .

C'est l'un des avantages de l'utilisation d'un framework comme DJANGO : toutes les fonctionnalités de persistance des données dans une base de données ont déjà été écrites pour vous. Tout ce que vous avez à faire est de faire en sorte que votre modèle hérite de la classe `models.Model` de DJANGO. Votre modèle hérite ensuite de toutes les méthodes (comportements) nécessaires pour effectuer des opérations telles que la sélection et l'insertion de données dans une base de données.

Vous n'aurez donc pas à écrire de code qui interagit avec une base de données, mais vous devrez apprendre à interagir avec les modèles de DJANGO pour faire la même chose (ne paniquez pas, c'est plus facile ! 😊).

Création du modèle

Créons maintenant notre premier modèle. Nous allons faire simple pour commencer. Un modèle est la source d'information unique et définitive pour vos données. Il contient les champs essentiels et le comportement attendu des données que vous stockerez. DJANGO respecte la philosophie DRY (Don't Repeat Yourself, «ne vous répétez pas»). Le but est de définir le modèle des données à un seul endroit, et ensuite de dériver automatiquement ce qui est nécessaire à partir de celui-ci.



Dans notre application, pour le moment, nous allons créer deux modèles : Produit et Catégorie. Un produit possède une référence, un intitulé et un prix unitaire. Une catégorie a un identifiant et un nom. Une catégorie est composée de plusieurs produits et un produit appartient qu'à une seule catégorie. Ces concepts sont représentés par des classes Python. Éditez le fichier `monApp/models.py` de façon à ce qu'il ressemble à ceci :

```

from django.db import models

class Categorie(models.Model):
    idCat = models.AutoField(primary_key=True)
    nomCat = models.CharField(max_length=100)

    def __str__(self):
        return self.nomCat

class Produit(models.Model):
    refProd = models.AutoField(primary_key=True)
    intituleProd = models.CharField(max_length=200)
    prixUnitaireProd = models.DecimalField(max_digits=10, decimal_places=2)
    # Relation CIF : chaque produit appartient à 1 catégorie (0,N côté catégorie → 1,1 côté produit)
    categorie = models.ForeignKey(Categorie, on_delete=models.CASCADE, related_name="produit_categorie", null=True,
    blank=True)

    def __str__(self):
        return self.intituleProd
  
```

Un modèle est donc une simple classe python qui hérite de la classe `models.Model`. Les champs sont définis dans la classe, on leur donne un nom et un type. Et voilà notre premier modèle.

Cela peut sembler différent des classes que vous avez construites auparavant. Les classes Python ont généralement un constructeur : la méthode `__init__`, où nous utilisons les arguments passés pour définir les valeurs des attributs d'instance. Avec les modèles DJANGO, les choses se font différemment. Le framework examine les champs du modèle (que nous définissons comme des attributs de classe), puis crée le constructeur pour nous. C'est un autre exemple de la «magie» du framework. Comme pour les classes Python, nous pouvons utiliser les modèles DJANGO pour créer des instances de cette classe.

Le champ des modèles DJANGO

Ici, chaque modèle est représenté par une classe qui hérite de `django.db.models.Model`. Chaque modèle possède des variables de classe, chacune d'entre elles représentant un champ de la base de données pour ce modèle.

<code>AutoField</code>	→ incrémente automatiquement sa valeur
<code>BinaryField</code>	→ stocke des données binaires brutes en octets (bytes)
<code>BooleanField</code>	→ un champ True / False
<code>CharField</code>	→ un champ pour une chaîne de caractères assez courte
<code>TextField</code>	→ pour du texte long
<code>CommaSeparatedIntegerField</code>	→ entiers séparés par un virgule
<code>EmailField</code>	→ vérifie une valeur d'adresse valide
<code>SlugField</code>	→ format slug (alphanumérique + tirets)
<code>URLField</code>	→ format URL
<code>DateTimeField</code>	→ une date, instance de <code>datetime.date</code> en python
<code>DateTimeField</code>	→ une date et une heure, instance python de <code>datetime.datetime</code>
<code>DecimalField</code>	→ un nombre décimal de taille fixe, instance python de <code>Decimal</code>
<code>FileField</code>	→ un champ de fichier à téléverser
<code>ImageField</code>	→ idem que <code>FileField</code> mais vérifie qu'il s'agit d'une image
<code>FilePathField</code>	→ un path de fichier (paramètre <i>path</i> est obligatoire)
<code>FloatField</code>	→ une instance de float en python
<code>GenericIPAddressField</code>	→ une adresse ip valide IPV4 / IPV6
<code>IPAddressField</code>	→ une adresse ip textuel type 192.168.0.1
<code>IntegerField</code>	→ valeurs comprises entre -2147483648 à 2147483647
<code>BigIntegerField</code>	→ Un entier 64 bits
<code>PositiveIntegerField</code>	→ valeurs comprises entre 0 et 2147483647
<code>PositiveSmallIntegerField</code>	→ valeurs comprises entre 0 et 32767
<code>SmallIntegerField</code>	→ valeurs comprises entre -32768 et 32767
<code>NullBooleanField</code>	→ un champ booléen qui accepte le Null
<code>TimeField</code>	→ format heure instance de <code>datetime.time</code>

Chaque champ est représenté par une instance d'une classe `Field` – par exemple, `CharField` pour les champs de type caractère, et `IntegerField` pour les champs de type entier. Cela indique à DJANGO le type de données que contient chaque champ. Le nom de chaque instance de `Field` est le nom du champ en interne. Vous l'utiliserez dans votre code Python et votre base de données l'utilisera comme nom de colonne.

Certaines classes `Field` possèdent des paramètres obligatoires. La classe `CharField`, par exemple, a besoin d'un attribut `max_length`. Ce n'est pas seulement utilisé dans le schéma de base de la base de données, mais également pour valider les champs, comme nous allons voir prochainement. Un champ `IntegerField` peut aussi autoriser des paramètres facultatifs (`default=0`)

Finalement, notez que nous définissons une relation, en utilisant `ForeignKey`. Cela indique à DJANGO que chaque élément (Item) n'est relié qu'à un seul produit (Product). DJANGO propose tous les modèles de

relations : plusieurs-à-un, plusieurs-à-plusieurs ([ManyToManyField](#)), un-à-un ([OneToOneField](#)), mais nous y reviendrons plus tard.

Les champs des modèles sont définis dans le module suivant: [django.db.models.fields](#). Il est donc possible de voir tous les champs existants en exécutant la commande [help\(django.db.models.fields\)](#)

----- paramètres communs -----

<code>db_column</code>	→ nom de la colonne dans la base de données
<code>db_index</code>	→ créer un index pour la colonne
<code>default</code>	→ la valeur par défaut du champ
<code>editable</code>	→ Si False le champ n'est pas éditable dans admin
<code>help_text</code>	→ texte d'aide affiché dans le formulaire
<code>primary_key</code>	→ si True devient la clé primaire
<code>unique</code>	→ si True impossible d'avoir des doublons de valeur
<code>verbose_name</code>	→ un nom plus explicite
<code>validators</code>	→ une liste de validateurs à exécuter

----- paramètres spécifiques -----

<code>primary_key</code>	→ renseigner la clé primaire
<code>blank</code>	→ autoriser la soumission d'un champ vide
<code>null</code>	→ autoriser d'enregistrer en base une valeur nulle
<code>unique_for_date</code>	→ unique pour une date
<code>unique_for_month</code>	→ unique pour un mois
<code>unique_for_year</code>	→ unique pour un an
<code>choices</code>	→ choix possibles

Migrations

Comme nous l'avons dit, une des caractéristiques d'un modèle est qu'il est capable de stocker ses données dans une base de données. Nous avons créé notre base de données dans le chapitre sur la configuration. Mais cette base de données ne sait encore rien de notre modèle. Et c'est là que les migrations entrent en jeu.

Si nous voulons stocker les produits dans notre base de données, nous aurons besoin d'une nouvelle table, contenant une colonne pour chaque champ que nous avons ajouté à notre modèle, ainsi qu'une colonne id pour servir de clé primaire : un identifiant unique pour chaque ligne de la table. La structure d'une base de données, en termes de tables et de colonnes, est appelée schéma.

Si nous construisions notre schéma de base de données manuellement, nous pourrions écrire une requête SQL ou utiliser une interface graphique de gestion de base de données, pour créer notre première table. Mais dans DJANGO, nous faisons les choses différemment. Nous utilisons une sous-commande de l'utilitaire de ligne de commande qui va générer des instructions pour construire la table. Et ensuite, nous utilisons une autre sous-commande pour exécuter ces instructions. Ces instructions sont appelées une migration.

Une migration est un ensemble d'instructions permettant de passer le schéma de votre base de données d'un état à un autre. Il est important de noter que ces instructions peuvent être exécutées automatiquement, comme un code.

En programmation, on parle souvent de «configuration par le code». Il s'agit d'une philosophie qui stipule que toutes les étapes nécessaires à la construction de votre application ne doivent pas être effectuées à la main, mais plutôt inscrites dans le code. Pourquoi ? Pour plusieurs raisons :

- Les étapes manuelles peuvent facilement être oubliées, mais les étapes écrites sous forme de code peuvent être stockées dans un repository afin qu'elles soient aussi sûres que tous vos autres codes sources.
- Lorsque les étapes sont conservées dans votre repository, elles peuvent être facilement partagées avec les autres membres de l'équipe.
- Les étapes écrites sous forme de code peuvent être exécutées automatiquement par votre ordinateur. Cette méthode est rapide et fiable, surtout s'il y a plusieurs étapes.

Maintenant que nous savons pourquoi les migrations sont importantes, créons-en une pour notre modèle. Exécutons une autre commande :

```
./manage.py makemigrations monApp
```

En exécutant **makemigrations**, vous indiquez à DJANGO que vous avez effectué des changements à vos modèles (dans ce cas, vous en avez créé) et que vous aimeriez que ces changements soient stockés sous forme de migration. Les migrations sont le moyen utilisé par DJANGO pour stocker les modifications de vos modèles (et donc de votre schéma de base de données), il s'agit de fichiers sur un disque.

Le résultat de la commande nous indique qu'une nouvelle migration a été enregistrée dans `monApp/migrations/0001_initial.py`, et que son objectif est de «Créer le modèle de produit et categorie», ce qui signifie en fait que cette migration va créer deux tables dans la base de données pour notre modèle. Vous pouvez consulter la migration pour vos nouveaux modèles si vous le voulez; il s'agit du fichier `monApp/migrations/0001_initial.py`. Soyez sans crainte, vous n'êtes pas censé les lire chaque fois que DJANGO en crée, mais ils sont conçus pour être humainement lisibles au cas où vous auriez besoin d'adapter manuellement les processus de modification de DJANGO.

L'avantage de cette commande est qu'elle analyse notre fichier `models.py` pour y déceler toute modification et déterminer le type de migration à générer. Maintenant que nous avons notre migration (nos instructions), nous devons exécuter ces instructions sur la base de données. Mais tout d'abord, voyons les instructions SQL que la migration produit.

La commande `sqlmigrate` accepte des noms de migrations et affiche le code SQL correspondant :

```
./manage.py sqlmigrate monApp 0001
```

La commande `sqlmigrate` n'exécute pas réellement la migration dans votre base de données - elle se contente de l'afficher à l'écran de façon à vous permettre de voir le code SQL que DJANGO pense nécessaire. C'est utile pour savoir ce que DJANGO s'apprête à faire ou si vous avez des administrateurs de base de données qui exigent des scripts SQL pour faire les modifications.

Si cela vous intéresse, vous pouvez aussi exécuter `./manage.py check`. Cette commande vérifie la conformité de votre projet sans appliquer de migration et sans toucher à la base de données.

Maintenant, exécutez à nouveau la commande `migrate` pour créer les tables des modèles dans votre base de données :

```
./manage.py migrate
```

Vous vous rappelez dans le chapitre sur l'installation quand nous avons ajouté «monApp» au `INSTALLED_APPS` de notre projet ? DJANGO a recherché dans chacune de ces applications installées de nouvelles migrations à exécuter, il a trouvé notre nouvelle migration et l'a «appliquée» : il a exécuté ces instructions sur la base de données.

La commande `migrate` sélectionne toutes les migrations qui n'ont pas été appliquées (DJANGO garde la trace des migrations appliquées en utilisant une table spéciale dans la base de données (`django_migrations`) puis les exécute dans la base de données, ce qui consiste essentiellement à synchroniser les changements des modèles avec le schéma de la base de données.

Les migrations sont très puissantes et permettent de gérer les changements de modèles dans le temps, au cours du développement d'un projet, sans devoir supprimer la base de données ou ses tables et en refaire de nouvelles. Une migration s'attache à mettre à jour la base de données en live, sans perte de données.

Nous les aborderons plus en détails dans une partie ultérieure de ce didacticiel, mais pour l'instant, retenez le guide en trois étapes pour effectuer des modifications aux modèles :

- Modifiez les modèles (dans `models.py`).
- Exécutez `manage.py makemigrations` pour créer des migrations correspondant à ces changements.
- Exécutez `manage.py migrate` pour appliquer ces modifications à la base de données.

La raison de séparer les commandes pour créer et appliquer les migrations est que celles-ci vont être ajoutées dans votre système de gestion de versions et qu'elles seront livrées avec l'application; elles ne font pas que faciliter le développement, elles sont également exploitables par d'autres développeurs ou en production.

C'est à vous de jouer !

- complétez votre modèle avec Rayon (migration 002)
- modifiez votre modèle tel qu'un produit possède une date de fabrication (migration 003)
- ajoutez une entité Statut composée d'un identifiant et d'un libelle (migration 004)
- modifiez votre modèle tel qu'un produit possède un statut (migration 005)

Faites plusieurs migrations en respectant bien les étapes et en observant les fichiers générés.

Tester le modèle

Le gabarit par défaut de `startapp` crée un fichier `tests.py` dans une application. Cela convient bien quand il n'y a que quelques tests. Mais dès que la suite de tests grandit, il devient préférable de restructurer ce fichier en un module Python afin de pouvoir séparer les tests en sous-modules tels que `test_models.py`, `test_views.py`, `test_forms.py`, etc. Vous êtes libre de choisir le découpage structurel qui vous convient.

Cependant, je vous invite à créer un répertoire `monApp/tests/`. Vous aurez besoin d'un fichier `__init__.py` dans ce sous-dossier `tests`. Aussi, dans ce répertoire `tests`, créez un fichier `test_View_Basics.py` dans lequel vous copierez le contenu du fichier de base `monApp/tests.py`. Vous pourrez supprimer le fichier `tests.py` ensuite.

Revenons au modèle. Créez le fichier `test_Model_Categorie.py` dans `monApp/tests/`. Autrement dit, nous allons tester le modèle de `Categorie`. Notre classe `CategorieModelTest` héritera de `TestCase`.

```
from django.test import TestCase
from monApp.models import Categorie
class CategorieModelTest(TestCase):
```

Une première méthode `setUp()` permet de créer les objets nécessaires pour chaque test. Dans cet exemple, une `categorie` est créée.

```
def setUp(self):
    # Créer un attribut produit à utiliser dans les tests
    self.ctgr = Categorie.objects.create(nomCat="CategoriePourTest")
```

Le test `test_categorie_creation()` vérifie que la création de l'objet `Categorie` se passe correctement.

```
def test_categorie_creation(self):  
    self.assertEqual(self.ctgr.nomCat, "CategoriePourTest")
```

Le test `test_string_representation()` vérifie que la méthode `__str__()` du modèle renvoie bien le format correct (comme vous l'avez défini dans votre modèle) :

```
def test_string_representation(self):  
    self.assertEqual(str(self.ctgr), "CategoriePourTest")
```

Le test `test_categorie_updating()` vérifie la capacité à mettre à jour un objet `Categorie` en changeant le nom et en s'assurant que la modification est bien enregistrée en base de données.

```
def test_categorie_updating(self):  
    self.ctgr.nomCat = "CategoriePourTestsModifiee"  
    self.ctgr.save()  
    # Récupérer l'objet mis à jour  
    updated_ctgr = Categorie.objects.get(idCat=self.ctgr.idCat)  
    self.assertEqual(updated_ctgr.nomCat, "CategoriePourTestsModifiee")
```

Le test `test_categorie_deletion()` vérifie que l'objet peut être supprimé, et qu'il n'est plus présent dans la base de données après la suppression.

```
def test_categorie_deletion(self):  
    self.ctgr.delete()  
    self.assertEqual(Categorie.objects.count(), 0)
```

Pour exécuter les tests, vous pouvez utiliser la commande suivante dans votre terminal à la racine de votre projet DJANGO :

```
./manage.py test monApp/tests
```

Et pour le coverage :

```
coverage run manage.py test monApp/tests  
coverage report
```

C'est à vous de jouer !

Si vous avez tout compris, testez le modèle `Produit`, `Rayon` et `Statut` afin de maintenir votre taux de couverture de test !

Remarque : Pour les champs `DateField`, `DateTimeField` et `TimeField`, il est recommandé de manipuler les types Python correspondants (`date`, `datetime`, `time`) plutôt que des chaînes de caractères. Django les convertit automatiquement lors de l'enregistrement et de la lecture en base de données.

Le shell Python et l'API DJANGO

Dans cette section, nous allons écrire du code dans le shell interactif Python pour jouer avec l'API que DJANGO met gratuitement à votre disposition. Le shell de DJANGO est simplement un shell Python ordinaire qui exécute votre application DJANGO. Vous pouvez le considérer comme un endroit où vous pouvez essayer du code en temps réel : chaque fois que vous appuyez sur Entrée, la ligne de code que vous venez de taper est exécutée. Ainsi, alors que le code que vous tapez dans un module/fichier Python peut être exécuté de nombreuses fois, le code que vous tapez dans le shell DJANGO n'est exécuté qu'une seule fois, puis oublié.

Utilisons le shell pour créer quelques objets, puis enregistrons ces objets dans la base de données. Vous pouvez utiliser mes exemples, ou n'hésitez pas à ajouter vos propres objets préférés ! Pour lancer un shell Python, utilisez cette commande : `python manage.py shell`

À l'invite du shell (`>>>`), tapez le code suivant pour importer notre modèle :

```
>>> from monApp.models import *  
Appuyez sur Entrée pour exécuter cette ligne.
```

Ensuite nous allons vérifier qu'il n'y a pas encore de produits dans notre base.

```
>>> Produit.objects.all()
```

Ensuite, nous allons créer une nouvelle instance du modèle Produit :

```
>>> prdt = Produit()  
>>> prdt.intituleProd="ipod"  
>>> prdt.prixUnitaireProd = 59.99
```

Jetez un coup d'oeil à l'état actuel de l'objet en tapant simplement `prdt` puis Entrée

```
>>> prdt
```

Le shell nous dit que nous avons un objet `prdt`, mais l'id est `None`, il n'a pas encore d'identifiant. Maintenant, sauvegardons cet objet dans la base de données :

```
>>> s=Statut(idStatut=0,libelleStatut="Offline")  
>>> s.save()  
>>> prdt.statut=s  
>>> prdt.save()
```

La méthode `save` enregistre les données dans la base. Tant que vous ne l'avez pas appelée, aucune information n'est enregistrée.

Chaque fois que nous insérons un objet dans la base de données, un identifiant est ajouté automatiquement pour nous. On aurait pu tout aussi bien créer une instance de `Produit` en utilisant la syntaxe suivante :

```
>>> Produit(intituleProd="iphone", prixUnitaireProd=1299.99, statut=s).save()
ou encore
>>> prdt=Produit.objects.create(intituleProd="ipad", prixUnitaireProd=499.69,statut=s)
```

Notre base de données contient maintenant 3 objets `Produit`. On peut vérifier ça comme ça :

```
>>> Produit.objects.count()
ou encore
>>> Produit.objects.all()
```

Pour l'instant, vous pouvez considérer qu'un `QuerySet` ressemble beaucoup à une liste Python. Profitons-en pour créer une catégorie de notre dernier produit (instance de `Categorie`) qu'on associe à notre dernier produit.

```
>>> ctgr= Categorie()
>>> ctgr.nomCat="Tablette"
>>> prdt.categorie=ctgr
>>> ctgr.save()
>>> prdt.save()
```

Vous pouvez récupérer des entrées de votre base de données (sous forme d'objet) via un manager que tout modèle possède et qui s'appelle `objects`. Nous l'avons utilisé dans les exemples précédent mais sans savoir ce qu'il est et ce qu'il est capable de faire. Vous pouvez d'ailleurs le voir dans le shell:

```
>>> Produit.objects
```

Ce manager possède un grand nombre de méthode qui vous permettra de filtrer les objets en fonction de votre besoin :

- Pour afficher toutes les entrées d'un modèle vous pouvez utiliser la méthode `all` :

```
>>> Produit.objects.all()
```

Le manager vous retourne une liste qui contient sous forme d'objet toutes les entrées de la base de données.

```
>>> Produit.objects.all()[0]
```
- Vous pouvez demander au manager de vous retourner un objet en utilisant la méthode `get` :

```
>>> Produit.objects.get(pk=2)
```
- La méthode `get` ne retourne qu'un seul objet, si vous voulez récupérer tous les objets ayant X conditions vous pouvez utiliser la méthode `filter`:

```
>>> Produit.objects.filter(prixUnitaireProd=59.99)
```
- Vous pouvez effectuer des recherches plus complexes comme "Commence par":

```
>>>Produit.objects.filter(intituleProd__startswith="ip")
```
- Le concept "Contient":

```
>>> Produit.objects.filter(intituleProd__icontains="ipad")
```

A noter que la recherche "contain" (au lieu de `icontains`) peut être aussi utilisé mais elle sera sensible à la casse.
- Quitter le shell : `exit()` ou `Ctrl+D` ou `quit()`

Affichage des données dans les vues

Alors, comment faire pour que nos objets sortent de la base de données et se retrouvent dans nos pages ? Première chose, récupérez le script `data.sql` sur CELENE pour alimenter votre BDD. Ensuite :

- créer une nouvelle fonction `ListProduits` dans votre fichier `views.py`
- nous avons vu comment obtenir tous les objets `Produit` de la base de données dans le shell : `Produit.objects.all()`.
Faites la même chose dans votre vue et stockez le résultat dans la variable `prdts`
- N'oubliez pas d'importer votre modèle `Produit` !
- La variable `prdts` contient maintenant une liste de tous les produits qui peuvent être trouvés dans la base de données. Cela signifie qu'il est maintenant possible d'accéder à chacun des objets `Produit` individuels en utilisant la notation d'index de Python, comme ceci :
`prdts[0]` # pour le premier objet `Produit`
`prdts[1]` # pour le second, etc.
- Vous ferez appel à l'attribut `intituleProd` de l'objet en utilisant la notation par points de Python, comme nous le ferions avec n'importe quel autre objet : `prdts[0].intituleProd`
- Utilisez ces techniques pour afficher les noms de vos produits dans notre page HTML. Utilisez les balises `<ul>` et `<li>` pour afficher sous forme de liste. A vous de construire la bonne chaîne HTML avant de la passer en argument de `HttpResponse` lors du `return`. Pour aider à la lisibilité, vous pouvez :
 - utiliser des guillemets triples (`"""`) pour répartir votre chaîne HTML sur plusieurs lignes ;
 - faire de cette chaîne une «f-string» (`f"""`) afin que nous puissions injecter nos noms de produit dans la chaîne en utilisant `{ ... }` .
- N'oubliez pas de modifier votre fichier `urls.py` pour associer votre nouvelle vue.
- Testez votre vue (avec et sans produits) pour maintenir le taux de couverture du fichier `views.py`

C'est à vous de jouer !

Refaites l'exercice pour les catégories de produits (objet `Categorie`), les rayons et les statuts (si vous avez bien réussi l'exercice tout à l'heure sur l'évolution du modèle...). N'oubliez pas les tests !